REDTAP CORE V1

Backend Security & Execution Governance Investigation

Technical Audit Report · Source-Based Assessment

Document status: Approved investigation — report issued
Repository: rokredess-droid/RedTap-Core-V1
Branch: master
Reviewed revision: 3681f19ad737b57e4d490a955529bb0a7ab14fb0
Assessment type: Static source-code review
Classification: Engineering / Security Architecture

Document Control

Field	Value
Subject	Authentication, authorization, tenant isolation and execution governance
System	RedTap Core V1
Evidence basis	Repository source files and visible integration-test files
Runtime validation	Not performed in this investigation
Intended audience	Platform architecture, backend engineering, security engineering
Decision context	Alignment with RedTap's frozen Security Law Book architecture

Executive Summary

This investigation examines whether RedTap Core V1's backend can reliably answer the execution-governance questions:

- Who acts?
- For whom does the actor act?
- Under which tenant and branch?
- On which resource and through which operation?
- Under which policy and authority?
- At which execution boundary is authority enforced?
- What effect was produced and how was it verified?
- What evidence is retained?

Executive conclusion

RedTap Core V1 contains a functional foundation for authentication and conventional tenant-scoped role-based access control (RBAC). JWT authentication, identity sessions, refresh-token rotation, permission claims, controller authorization attributes, tenant-scoped service queries and security-event records are present.

The reviewed implementation does **not yet constitute a unified execution-governance architecture** corresponding to the frozen reference:

> **Security Law Book → Dynamic Authority Derivation → Portable Authority → Independent Enforcement at Every Execution Boundary → Verified Effect → Audit**

The gap is architectural rather than a reason to replace the existing identity foundation. The appropriate direction is to preserve JWT, `ICurrentUser`, RBAC and Clean Architecture, then strengthen security invariants and introduce an explicit execution-authority layer.

Priority conclusions

1. **Access-token revocation:** session revocation is not demonstrated to invalidate an already-issued access JWT at request authorization.
2. **Tenant-qualified login:** the database permits tenant-local usernames, while login lookup is username-only.
3. **Branch integrity:** user creation/update accepts branch identifiers without demonstrated tenant ownership validation.
4. **Relationship integrity:** application checks exist, but inspected relational configurations do not enforce tenant-consistent user-role relationships.
5. **Authorization freshness:** permissions are embedded in JWT claims; existing tokens can retain permissions after role changes until another validity mechanism intervenes.
6. **Module execution authority:** module services accept tenant identifiers and lack a common service-level authority boundary in the inspected code.
7. **Provisioning and vouchers:** tenant provisioning is a transaction skeleton; voucher redemption is unimplemented.
8. **Event execution:** hosted inbox/outbox workers contain placeholder behavior and do not establish completed event delivery/processing.
9. **Execution evidence:** identity/security telemetry exists, but a general action-to-effect audit and verification contract was not identified.

These findings require remediation and tests before the corresponding properties can be treated as security guarantees.

---

1. Scope, Method and Evidence Standard

1.1 Scope

The source review covered representative backend components:

- API authentication and middleware configuration
- Current-user identity abstraction
- JWT generation and permission claims
- Login and identity-session services
- Refresh-token rotation and replay detection
- Permission authorization handler and policy provider
- User, role and permission management services
- Module registry, manifest, installation and lifecycle services
- License, voucher and tenant-provisioning services
- Inbox/outbox and hosted worker foundations
- Notification pipeline and audit service
- EF Core entity configurations and relationships
- Visible integration tests for identity sessions and logout

1.2 Method

The investigation used source inspection to trace:

1. Where identity and tenant context originate.
2. Where authorization decisions are made.
3. Whether tenant and branch ownership are checked.
4. Whether revocation affects access-token validation.

5. Whether service operations independently enforce authority.
6. Whether state changes are verified.
7. Whether execution evidence is recorded.
8. Whether database relationships reinforce application invariants.
9. Whether background execution has explicit identity and authority.
10. Whether available tests demonstrate the relevant security properties.

1.3 Classification vocabulary

Classification	Meaning
IMPLEMENTED	Direct implementation is visible in the reviewed source
PARTIAL	Some control exists, but its scope or completeness is limited
MISSING / NOT IDENTIFIED	No implementation was identified in the reviewed source scope
INCOMPLETE	Implementation explicitly remains a skeleton or placeholder
UNVERIFIED	Source inspection alone cannot establish runtime behavior

Important: "Not identified" is not a claim that no code exists anywhere in the repository. It means the reviewed source set did not establish the capability. Runtime and repository-wide follow-up may change a classification.

---

2. Current Identity and Authentication Architecture

2.1 Identity context

The `ICurrentUser` contract exposes:

- `IsAuthenticated`
- `UserId`
- `TenantId`
- `BranchId`
- `SessionId`
- `Username`

The infrastructure implementation resolves these values from authenticated HTTP claims, including the user identifier, tenant, branch and session identifiers.

Assessment: The interface is a useful request identity abstraction. It identifies the authenticated actor and declared scope; it is not a complete execution context or authority decision.

2.2 JWT composition

The reviewed JWT service includes claims for:

- User/subject identifier
- Username
- Session identifier
- Tenant identifier
- Branch identifier
- Permission codes

The API validates JWT signature, issuer, audience and lifetime, and uses the authentication/authorization middleware pipeline.

Assessment: This is a conventional and useful token-based authentication foundation. Permission claims provide portable authorization data, but their presence does not by itself establish current authority for every resource or execution context.

2.3 Login and permission derivation

Login loads user role memberships and associated role permissions. The permission set is derived through:

```
UserRoles → Roles → RolePermissions → Permissions
```

The resulting permissions are passed into JWT generation and session-related flows.

The inspected login query searches by username without tenant qualification. The database configuration, however, permits username uniqueness within a tenant rather than globally.

Finding SEC-02 — Tenant-qualified login ambiguity

- **Status:** Confirmed in reviewed source
- **Risk:** High
- **Condition:** Identical usernames may exist in different tenants
- **Required control:** Deterministic tenant resolution before selecting the account
- **Regression proof:** Duplicate username across two tenants must authenticate only in the intended tenant context

The platform must explicitly select a canonical identity-resolution strategy. Tenant-qualified login is the recommended direction, subject to product and identity design decisions.

---

3. Session Revocation and Access-Token Validity

3.1 Observed behavior

The identity-session service manages session records, refresh-token hashes, expiry and revocation. The session activity middleware updates activity for the session associated with a request.

The reviewed middleware does not demonstrate a request rejection when the referenced session is revoked, expired or absent. Its activity update path can return without updating a revoked or missing session and then continue the middleware pipeline.

JWT bearer validation does not, in the inspected configuration, consult the session record as part of access-token acceptance.

Finding SEC-01 — Access JWT survives session revocation

- **Status:** Confirmed in reviewed source
- **Risk:** Critical
- **Affected controls:** Logout, logout-all, administrative revocation, refresh-token replay response
- **Security distinction:** Revoking a refresh-token session is not equivalent to invalidating an already-issued access token

An access token can remain cryptographically valid until its expiry even when its session has been revoked, unless another request-time control rejects it.

Required invariant

[Session.RevokedAt eq null Rightarrow Request.Authorization = Deny]

The exact enforcement design may use authoritative lookup or a cache with rigorous invalidation semantics. It must not rely on activity tracking as a revocation check.

Required regression tests

- A valid access token is accepted before revocation.
- The same token is rejected after session revocation.
- Logout invalidates access according to the documented contract.
- Logout-all invalidates all affected sessions.
- Revoked or missing session IDs are rejected.
- Session expiry is enforced independently of JWT expiry.
- Revocation behavior remains correct with cached session state.

3.2 Refresh rotation and concurrency

Refresh-token hashing, rotation and replay detection are present. The refresh flow reloads role/permission data before issuing a new access token.

The reviewed source does not establish an atomic single-use refresh protocol under simultaneous requests.

Required verification: concurrent refresh requests against one token must produce a deterministic winner/rejection outcome; replay detection and session state must remain consistent.

---

4. Tenant and Branch Isolation

4.1 Tenant-scoped service operations

The user and role services generally scope operations to the current tenant:

- User listing and retrieval
- User updates
- Role creation and listing
- Role updates and deletion
- Role assignment and removal
- Role-permission operations

These controls are valuable and should be preserved.

4.2 Tenant isolation is query-dependent

The reviewed code primarily enforces tenant isolation through individual service queries. A universal mechanism that automatically protects every tenant-owned entity and every new query was not established.

This creates a maintenance risk: every future module, service, background task and data access path must remember to apply the correct tenant predicate.

4.3 Branch ownership

User creation derives tenant identity from the current user but accepts a branch identifier from the request. User update also accepts a branch identifier.

The reviewed service logic does not demonstrate a mandatory branch-to-tenant ownership check for these operations. A foreign branch identifier may therefore be accepted if it exists and no other constraint blocks it.

Finding SEC-03 — Branch ownership validation gap

- **Status:** Confirmed in reviewed source
- **Risk:** High
- **Required control:** Validate branch ownership before user creation or branch change
- **Regression proof:** A tenant cannot assign its user to a branch belonging to another tenant

4.4 Ownership invariants

For tenant-owned resources:

[Resource.TenantId = ExecutionContext.TenantId]

For branch-scoped resources:

[Resource.BranchId in Branches(ExecutionContext.TenantId)]

The invariant should be enforced at application boundaries and reinforced through relational constraints where feasible.

---

5. RBAC and Authorization Freshness

5.1 Current mechanism

The permission authorization handler checks whether the required permission code exists in the authenticated token's permission claims. Dynamic policy names are translated into permission requirements, and controller attributes declare required permissions.

This provides declarative endpoint protection and a clear permission vocabulary.

5.2 Authorization snapshot

The effective permissions are derived at login or refresh and embedded in the JWT.

Consider the sequence:

1. User receives permission P in a token.
2. Administrator removes P from the user's role.
3. The existing access token still contains P.
4. Endpoint authorization checks the token claim.

Unless token/session invalidation or another current-authority check intervenes, the existing token may continue to satisfy the permission requirement.

Finding SEC-05 — Permission freshness is token-bound

- **Status:** Confirmed architectural characteristic
- **Risk:** High architectural concern
- **Required decision:** Define permission-change propagation and revocation semantics
- **Recommendation:** Keep JWT claims as an optimization, not the sole final authority for sensitive execution

5.3 Service-level enforcement

Controller attributes protect the API surface. They do not automatically protect application services invoked by background jobs, internal workflows, tests, future APIs or other callers.

The reviewed architecture does not demonstrate a universal service-level execution authorization contract.

Recommendation: Preserve endpoint RBAC and introduce a shared authorization boundary for sensitive application operations. The boundary should evaluate an explicit execution context and resource/action policy.

---

6. Role and Permission Relationship Integrity

The EF Core configurations define user-role and role-permission relationships with composite keys and foreign keys.

Role assignment service logic validates tenant ownership. However, the inspected relationship configuration does not itself enforce that a user and role in a UserRole relationship share the same tenant.

Finding SEC-06 — Tenant consistency is not structurally guaranteed by inspected relationship configuration

- **Status:** Confirmed configuration gap
- **Risk:** High architectural concern
- **Required control:** Tenant-consistent relationship integrity
- **Regression proof:** Cross-tenant role membership cannot be persisted through any supported write path

Application authorization and database integrity are complementary. The database should prevent inconsistent ownership relationships even when a future service or alternate write path omits validation.

---

7. Module Lifecycle and Execution Authority

7.1 Module foundation

The backend contains foundations for:

- Registry
- Manifest validation
- Installation
- Dependency graph
- Compatibility
- Upgrade
- Uninstallation
- Lifecycle operations

The module installation configuration includes tenant ownership and a uniqueness constraint over tenant and module registry identity.

7.2 Authority boundary

The reviewed module operations accept tenant identifiers as method/request parameters. A common service-level mechanism establishing that the caller may act for the requested tenant was not identified.

This is especially important because module services may later be invoked through:

- HTTP endpoints
- Administrative automation
- Background jobs
- Event consumers
- Module-to-platform calls

Finding SEC-04 — Module tenant authority is caller-dependent

- **Status:** Confirmed in reviewed service design
- **Risk:** High
- **Required control:** Explicit actor, tenant, module resource and lifecycle action authorization
- **Regression proof:** A caller authorized for tenant A cannot install, enable, upgrade or uninstall a module in tenant B

7.3 Lifecycle completeness

The reviewed lifecycle publish operation is a completed-task placeholder. The module foundation should not be represented as a fully operational lifecycle execution system until state transitions, authorization, event delivery, verification and audit are implemented and tested.

---

8. Licensing and Tenant Provisioning

8.1 License service

License operations provide a service foundation, but the inspected service methods do not independently establish caller authority.

Privileged licensing operations require explicit Super Admin authorization and an auditable execution boundary.

8.2 Voucher redemption

Voucher redemption remains unimplemented in the reviewed source.

8.3 Tenant provisioning

The provisioning service opens a database transaction but contains implementation placeholders for:

- Validation
- License
- Subscription
- Tenant
- Branch
- Administrator role
- Administrator user
- Permission assignment
- Audit

It returns a pending-implementation status.

Finding SEC-09 — Provisioning workflow incomplete

- **Status:** Confirmed
- **Impact:** Functional blocker
- **Required control:** Transactional provisioning, privileged authority, license validation, ownership invariants, verified completion and audit

Provisioning must be treated as a privileged workflow with explicit authority and postcondition checks—not merely a sequence of inserts.

---

9. Events, Background Execution and Notifications

9.1 Event infrastructure

The backend contains event envelopes, serializers, registry, inbox/outbox abstractions, retry strategies and hosted services.

The reviewed hosted workers contain placeholder behavior:

- Outbox dispatch retrieves pending events and marks them processed without demonstrating transport publication in the worker.
- Inbox processing retrieves pending events and marks them processed without demonstrating actual event handling.

Finding SEC-10 — Event execution is incomplete

- **Status:** Confirmed in reviewed source
- **Impact:** Functional blocker
- **Risk:** Processed-state semantics may not represent actual delivery or execution

The worker must not record successful processing before the relevant effect has occurred and been verified.

9.2 Background identity

Background execution cannot assume an HTTP request's `ICurrentUser` context is available.

The future model must distinguish:

- Initiating actor
- System/worker identity
- Event producer
- Event consumer
- Tenant scope
- Resource/action
- Delegation or execution grant
- Policy decision
- Correlation and causation

A background worker should execute under explicit system authority constrained by the event and its originating authorization context—not as an implicit superuser.

9.3 Notifications

The notification foundation includes pipeline, dispatcher, policy, recipient resolver and audit service.

The inspected defaults include an allow-all notification policy, null channel and empty recipient resolver. These are scaffolding defaults, not production delivery controls.

Notification audit records processing outcome, but they do not replace general execution audit.

---

10. Audit and Verified Effect

10.1 Existing telemetry

Security-event records include:

- Tenant
- User

- Event type
- Description
- Device
- IP address
- User agent
- Timestamp

Notification audit records include notification identity/type, success, error and processing time.

These are useful operational/security records.

10.2 General execution evidence

A universal execution record was not identified in the reviewed implementation that binds:

- Actor
- Subject
- Delegation chain
- Tenant/branch
- Resource
- Action
- Authority derivation
- Policy reference/version
- Decision
- Execution attempt
- Result
- Verification outcome
- Rejection/failure reason
- Correlation/causation identity

Architectural distinction: An authentication event proves an identity event occurred. It does not prove that a particular operation was authorized, executed correctly or produced its intended effect.

10.3 Required audit lifecycle

A future execution record should capture the complete lifecycle:

[Request ightarrow Decision ightarrow Execution ightarrow Verification ightarrow Audit]

The audit evidence should be durable, correlated and attributable. Sensitive values should be minimized; secrets, raw refresh tokens and credentials must not be recorded.

---

11. Consolidated Findings Register

ID	Finding	Classification	Severity / priority	Required response
SEC-01	Session revocation does not demonstrably invalidate access JWTs	Confirmed	Critical / P0	Enforce session validity at request authorization
SEC-02	Username-only login conflicts with tenant-local uniqueness	Confirmed	High / P0	Define tenant-qualified identity resolution
SEC-03	Branch ownership is not validated for user create/update	Confirmed	High / P0	Validate branch belongs to current tenant

ID	Finding	Classification	Severity / priority	Required response
SEC-04	Module service authority is caller-dependent	Confirmed	High / P1	Introduce module execution authorization
SEC-05	Permission claims are authorization snapshots	Confirmed	High / P1	Define permission freshness/revocation
SEC-06	User-role tenant consistency is not structurally enforced	Confirmed	High / P1	Add tenant-consistent relational constraints
SEC-07	Universal resource/context policy evaluator not identified	Gap	High / P1	Define execution authority contract
SEC-08	General action/effect audit not identified	Gap	High / P1	Define execution evidence model
SEC-09	Tenant provisioning remains a skeleton	Confirmed	Functional / P1	Complete privileged transactional workflow
SEC-10	Inbox/outbox workers contain placeholder processing	Confirmed	Functional / P1	Implement delivery/processing and truthful state transitions
SEC-11	Refresh-token concurrency guarantees unverified	Unverified	P1	Add concurrency and replay tests
SEC-12	General effect verification not identified	Gap	High / P1	Define postcondition verification contracts

Priority definitions: P0 = security correctness before broader rollout; P1 = foundational control before expanding affected capabilities. Severity and implementation priority are related but not identical.

---

12. Alignment With the Frozen Security Law Book Architecture

Reference architecture

Security Law Book → Dynamic Authority Derivation → Portable Authority → Independent Enforcement at Every Execution Boundary → Verified Effect → Audit

Reference component	Current implementation	Assessment
Security Law Book	No central law/policy evaluator identified	Missing
Dynamic authority derivation	Role/permission derivation at login/refresh	Partial
Portable authority	JWT identity and permission claims	Partial
Independent enforcement	Controller attributes and service-specific checks	Partial
Every execution boundary	No universal mechanism across HTTP, modules and workers	Missing
Verified effect	No general postcondition verification contract	Missing
Audit	Identity/security telemetry; no universal execution evidence	Partial

Architectural interpretation

The existing JWT and RBAC layers should remain useful components of the future system.

The change is not to replace authentication with policy evaluation. It is to separate:

1. **Identity:** who authenticated?
2. **Representation:** for whom or what is the actor acting?
3. **Authority:** what may this execution do under applicable constraints?

4. **Enforcement:** where is the decision independently enforced?
5. **Effect:** what actually happened?
6. **Verification:** did the intended effect occur?
7. **Evidence:** what can be independently audited?

---

13. Remediation Roadmap

Track A — Security correctness (P0)

1. Add request-time session validity enforcement.
2. Define tenant-qualified login resolution.
3. Validate branch ownership on user create/update.
4. Add tenant-consistent relationship constraints.
5. Specify permission-change and access-token revocation semantics.
6. Test concurrent refresh and replay behavior.

Exit criteria: Security invariants are demonstrated by automated tests; no cross-tenant or revoked-session acceptance remains in the covered paths.

Track B — Execution authorization foundation (P1)

1. Define immutable execution-context contract.
2. Separate actor from represented subject.
3. Define delegation representation and constraints.
4. Define tenant, branch, resource and action identifiers.
5. Define authority derivation inputs and provenance.
6. Define policy decision semantics: permit, deny, escalate.
7. Define service-level enforcement contract.
8. Preserve endpoint RBAC as an entry-point control.

Exit criteria: Sensitive operations can be authorized independently of whether they were invoked through an HTTP controller.

Track C — Execution-boundary coverage (P1)

Apply the common contract to:

- HTTP controllers and application services
- Module installation/lifecycle
- Licensing and provisioning
- Event consumers and background workers
- Notification execution
- Future AI and device commands

Exit criteria: Every identified execution path has an explicit identity, tenant/resource scope, authorization decision and rejection behavior.

Track D — Effect verification and audit (P1)

1. Define execution record schema.
2. Correlate request, decision, execution and result.
3. Define resource-specific postconditions.
4. Record verification success/failure.

5. Record denial and escalation evidence.
6. Define audit integrity, retention and sensitive-data minimization.

Exit criteria: A reviewer can reconstruct who initiated an operation, what authority was applied, what effect occurred and whether the result was verified.

Track E — Functional foundations

Complete and test:

- Tenant provisioning
- Voucher redemption
- Event publication and processing
- Notification delivery
- Module lifecycle state transitions

These should adopt the authority and audit contracts rather than introduce independent security models.

---

14. Regression and Verification Plan

The following tests should be treated as security-property tests, not merely unit coverage.

Test family	Required assertion
Session revocation	Revoked session cannot authorize requests with prior JWT
Session expiry	Expired session is rejected regardless of token state
Tenant login	Duplicate usernames across tenants resolve deterministically
Tenant isolation	Tenant A cannot read or mutate Tenant B resources
Branch isolation	Branch assignment cannot cross tenant ownership
Role integrity	Cross-tenant user-role relationship cannot be persisted
Permission freshness	Revoked permission cannot be used beyond documented invalidation contract
Service authorization	Direct service invocation enforces authority
Module lifecycle	Cross-tenant install/enable/upgrade/uninstall is rejected
Provisioning	Only authorized actor can provision; resulting tenant state is verified
Event processing	Event is not marked processed before successful processing
Background authority	Worker executes only within explicit tenant/resource constraints
Audit	Every sensitive execution has correlated decision and outcome evidence
Effect verification	Failed postcondition is recorded as failure, not success
Refresh concurrency	Simultaneous refresh produces deterministic single-use behavior

Verification method

For each test family:

1. Establish precondition and actor context.
2. Attempt permitted and prohibited operation.
3. Observe API/service decision.
4. Inspect persisted state.
5. Inspect audit evidence.
6. Verify no unauthorized side effect occurred.

7. Repeat through alternate execution paths where applicable.

---

15. Investigation Limitations and Evidence Boundaries

This report is based on static source inspection of the reviewed files and the visible identity integration-test files.

It does **not** claim that the following have been performed:

- Full repository-wide endpoint and call-graph enumeration
- Build or test execution
- Live PostgreSQL constraint verification
- Runtime session-revocation experiment
- Penetration testing
- Race-condition testing
- Deployment configuration review
- Secret-management review
- Complete migration-history analysis
- Review of every future module or execution path

Accordingly:

- Confirmed findings are tied to inspected source behavior/configuration.
- Missing capabilities are described as “not identified” where repository-wide proof is not available.
- Runtime consequences should be validated through dedicated tests before final security sign-off.
- The reviewed commit identifies the evidence baseline; findings must be rechecked against later revisions before implementation closure.

---

16. Final Engineering Judgment

RedTap Core V1 has a sound direction for an identity and RBAC foundation, but it should not yet be described as implementing the complete Security Law Book execution-governance model.

The highest-value path is:

1. **Correct session and tenant/branch security invariants.**
2. **Preserve current authentication, JWT and RBAC architecture.**
3. **Introduce explicit execution context and authority derivation.**
4. **Enforce authority independently at every execution boundary.**
5. **Verify effects and produce correlated audit evidence.**
6. **Apply the same contract to modules, events, AI, devices and connectors.**

This sequence preserves architectural continuity and avoids a premature rewrite.

Final statement

> RedTap Core V1 currently establishes authenticated identity and conventional permission-based access. The next architectural maturity step is to make authority contextual, execution-boundary independent, revocable, verifiable and auditable—without weakening or replacing the existing identity foundation.

Appendix A — Reviewed Source Index

Repository: RedTap-Core-V1

Identity and authorization

- backend/src/RedTap.Application/Security/ICurrentUser.cs
- backend/src/RedTap.Infrastructure/Security/CurrentUser.cs
- backend/src/RedTap.Infrastructure/Authentication/JwtService.cs
- backend/src/RedTap.Infrastructure/Authentication/LoginService.cs
- backend/src/RedTap.Infrastructure/Authentication/IdentitySessionService.cs
- backend/src/RedTap.Infrastructure/Authentication/PermissionAuthorizationHandler.cs
- backend/src/RedTap.Infrastructure/Authentication/PermissionPolicyProvider.cs
- backend/src/RedTap.API/Middleware/IdentitySessionActivityMiddleware.cs
- backend/src/RedTap.API/Program.cs

User, role and permission management

- backend/src/RedTap.Infrastructure/UserManagement/UserService.cs
- backend/src/RedTap.Infrastructure/RoleManagement/RoleService.cs
- backend/src/RedTap.Infrastructure/PermissionManagement/PermissionService.cs
- backend/src/RedTap.API/Controllers/UsersController.cs
- backend/src/RedTap.API/Controllers/RolesController.cs
- backend/src/RedTap.API/Controllers/PermissionsController.cs

Modules, licensing and provisioning

- backend/src/RedTap.Infrastructure/Modules/ModuleRegistryService.cs
- backend/src/RedTap.Infrastructure/Modules/ModuleManifestService.cs
- backend/src/RedTap.Infrastructure/Modules/ModuleInstallationService.cs
- backend/src/RedTap.Infrastructure/Modules/Lifecycle/ModuleLifecycleService.cs
- backend/src/RedTap.Infrastructure/Modules/ModuleUpgradeService.cs
- backend/src/RedTap.Infrastructure/Licensing/LicenseService.cs
- backend/src/RedTap.Infrastructure/Licensing/SubscriptionService.cs
- backend/src/RedTap.Infrastructure/Licensing/VoucherService.cs
- backend/src/RedTap.Infrastructure/Provisioning/TenantProvisioningService.cs

Events, notifications and persistence

- backend/src/RedTap.Infrastructure/Events/EventProcessor.cs
- backend/src/RedTap.Infrastructure/Events/InboxService.cs
- backend/src/RedTap.Infrastructure/Events/OutboxService.cs
- backend/src/RedTap.Infrastructure/BackgroundServices/InboxProcessorHostedService.cs
- backend/src/RedTap.Infrastructure/BackgroundServices/OutboxDispatcherHostedService.cs
- backend/src/RedTap.Infrastructure/Notifications/NotificationPipeline.cs
- backend/src/RedTap.Infrastructure/Notifications/NotificationAuditService.cs
- backend/src/RedTap.Infrastructure/Persistence/ApplicationDbContext.cs
- backend/src/RedTap.Infrastructure/Persistence/Configurations/UserConfiguration.cs
- backend/src/RedTap.Infrastructure/Persistence/Configurations/RoleConfiguration.cs
- backend/src/RedTap.Infrastructure/Persistence/Configurations/BranchConfiguration.cs
- backend/src/RedTap.Infrastructure/Persistence/Configurations/UserRoleConfiguration.cs
- backend/src/RedTap.Infrastructure/Persistence/Configurations/RolePermissionConfiguration.cs
- backend/src/RedTap.Infrastructure/Persistence/Configurations/ModuleInstallationConfiguration.cs

Tests reviewed

- `backend/tests/RedTap.IntegrationTests/Authentication/IdentitySessionTests.cs`
- `backend/tests/RedTap.IntegrationTests/Authentication/LogoutTests.cs`

Appendix B — Architectural Terms

Term	Meaning in this report
Actor	Authenticated identity initiating an operation
Subject	Person/entity represented by or affected by the operation
Delegation	Explicit authority to act for another subject
Execution context	Actor, subject, tenant, resource, action and relevant context
Authority	Constraints and permissions applicable to the execution
Policy decision	Permit, deny or escalate outcome
Execution boundary	Point where an operation can cause a state change or external effect
Verified effect	Evidence that the intended postcondition actually occurred
Execution audit	Correlated evidence of request, decision, effect and verification

End of report